Final Report Machine Learning for Systems and Control

1st Leliveld, Joost
Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands
j.j.p.leliveld@student.tue.nl

2nd Hocks, Timo
Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands
t.k.hocks@student.tue.nl

3rd Berg, Dirk van den
Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands
d.j.l.m.v.d.berg@student.tue.nl

4th Lande, Maurits van de
Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands
m.v.d.lande@student.tue.nl

I. Introduction

Modern control of non-linear dynamical systems increasingly relies on capturing unknown relationships from data combined with learning-based policies to complement or replace first-principles models. Machine learning methods such as Gaussian processes and neural networks can be used as general function approximators. These methods can be used to learn a map from the response of the system to actuation inputs to achieve a desired task. To achieve this a model needs to be built up by observing data from results of experiments. In this project we consider the classic unbalanced disk–pendulum setup. The primary object is to obtain a controller for the system that can swing up the pendulum and keep it stable at the top position. To achieve this we address the following challenges:

1) **Dynamics identification.** Obtain a mapping from past voltages and angles to future angles using Gaussian Process (GP) as well as Artificial Neural Network (ANN) models. The model dynamics are critical for a model-based policy learning approach.
2) **Policy learning.** Design a feedback controller that *swing-ups* the pendulum from its stable downward equilibrium to the upright position and is able to keep it there.

By integrating GP- and ANN-based identification with several reinforcement-learning techniques, we will investigate the trade-offs between probabilistic interpretability, data efficiency and control performance under realistic experimental conditions for different methods.

II. System identification

The first step in our study is to identify a data-driven model of the disk–pendulum dynamics. The ideal motion dynamics of the inverted pendulum are given as:

$$\dot{\theta} = \omega, \qquad \dot{\omega} = -\omega_0^2 \sin\theta \; - \; \gamma\,\omega \; - \; F_c \,\mathrm{sign}(\omega) \; + \; K_u\, u,$$

In practice the constants ω_0, γ, F_c and K_u cannot be determined with high accuracy. Coulomb friction (F_c) introduces discontinuous jumps in the torque, while the motor voltage is software-limited to ±3 V, causing saturation that violates the small-signal assumptions of the ideal model. Moreover, unmodeled effects such as stiction, structural flexibilities, and unsteady airflow manifest as higher-order dynamics that the simple four-parameter model cannot represent. As a result, a purely first-principles approach yields residual errors and poor long-horizon predictions.

A. Gaussian-Process Model Identification

In this part, the (non-linear) system dynamics will be captured using Gaussian Process (GP) regression. GPs offer a non-parametric Bayesian framework with the ability to tell how confident each prediction is. To model pendulum dynamics a Nonlinear AutoRegressive with eXogenous inputs (NARX) model was chosen. To make efficient use of the 35,000 data points, a sparse GP using the Nyström method is chosen, as discussed in lecture 5.

To capture the pendulum's periodic motion, we looked at two distinct feature representations to capture these dynamics: (i) raw features $[u[k-n_b], \ldots, u[k-1], \theta[k-n_a], \ldots, \theta[k-1]]$ and (ii) trigonometric features $[u[k-n_b], \ldots, u[k-1], \sin(\theta[k-n_a]), \ldots, \sin(\theta[k-1]), \cos(\theta[k-n_a]), \ldots, \cos(\theta[k-1])]$. Each representation was tested for multiple kernel types (RBF, Matérn 1.5, Matérn 2.5, and Periodic), the optimal kernel was determined with the use of Bayesian optimization. Since the test submission format is suited to the raw angle predictions, for implementation we focus on the raw features.

1) Model Selection for Pendulum Dynamics: We considered several structures for modeling the nonlinear dynamics of the pendulum system, each with their own advantages and disadvantages. NARX models allow for simplistic uncertainty propagation, while also providing stable training by relying on ground truth measurements. A NOE model was attempted but deemed to computational expensive for the purpose of GP. Unlike NARX which needs measured outputs NOE needs predicted outputs, this recursive dependency requires iterative optimization. For simulation NOE models outperform NARX models, due to NARX models accumulating prediction errors

over time.

However, NARX is the better fit for this pendulum system with strong dependence on recent angular positions and control inputs. It properly balances capturing system dynamics with computational efficiency.

2) The Data-Driven Modeling Process: The dataset of 35,000 samples is split as: 60% training (21,000 samples), 20% validation (7,000 samples), and 20% test (7,000 samples). We make use of two stage optimization. Memory depth optimization, where we look at the optimal the number of inputs and outputs to for the model. *TimeSeriesSplit* cross-validation with 3 folds is applied to maintain time-dependent causality, this makes sure that future data has no impact on past predictions.

The GP-NARX model applies Nyström approximation with 1,500 components (7% of training data) and the Scikit's kernel approximation was combined with Ridge regression. Then the validation RMSE for one-step prediction was minimized over 25 trials by altering kernel type selection, length scales, signal variance, noise level, and mean function type using Bayesian optimization.

We evaluate the GP-NARX model on the test set for one-step prediction and and multi-step simulation, measured in RMSE (radians).

3) Mathematical Formulation: Our NARX implementation for modeling the pendulum dynamics:

$$\theta[k] = f([u[k-n_b], \ldots, u[k-1], \theta[k-n_a], \ldots, \theta[k-1]]) + \varepsilon[k] \tag{1}$$

where $f \sim \mathcal{GP}(m(x), k(x, x'))$ with composite kernel:

$$k(x, x') = \sigma_f^2 k_{\text{base}}(x, x') + \sigma_n^2 \delta_{x,x'} \tag{2}$$

Function f is modeled with Nyström-accelerated regression for the Gaussian Process. We transform the features with $m = 1,500$ components:

$$\Phi = \text{Nyström}(X) \in \mathbb{R}^{N \times m} \tag{3}$$

Afterwards we apply Ridge regression:

$$\hat{\mathbf{w}} = (\Phi^T \Phi + \alpha I)^{-1} \Phi^T \mathbf{y}_{\text{normalized}} \tag{4}$$

$$\hat{y} = \Phi_{\text{test}} \hat{\mathbf{w}} \tag{5}$$

here is α the regularization parameter (with 10^{-3}).

This sparse approximation reduces computational complexity from $O(N^3)$ to $O(Nm^2 + m^3)$.

We consider the following base kernels:

- RBF: $k_{\text{RBF}}(x, x') = \exp\left(-\frac{\|x-x'\|^2}{2\ell^2}\right)$
- Matérn with $\nu \in \{1.5, 2.5\}$: $k_{\text{Matérn}}(x, x') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\|x-x'\|}{\ell}\right)^{\nu} K_\nu \left(\frac{\sqrt{2\nu}\|x-x'\|}{\ell}\right)$
- Periodic: $k_{\text{Per}}(x, x') = \exp\left(-\frac{2\sin^2(\pi\|x-x'\|/p)}{\ell^2}\right)$

Allowing the kernel to model the following behavior:

- $\sigma_f^2 k_{\text{base}}$: Models the smooth, predictable part of the signal
- $\sigma_n^2 \delta_{x,x'}$: WhiteKernel models measurement noise
- ConstantKernel(σ_f^2): Allows the model to adjust the signal amplitude

We compare two mean functions during optimization:

$$\text{Target modeling} = \begin{cases} y = f_{\text{GP}}(x) + \varepsilon & \text{(zero mean)} \\ y = f_{\text{linear}}(x) + f_{\text{GP}}(x) + \varepsilon & \text{(linear mean)} \end{cases} \tag{6}$$

where we fit $f_{\text{linear}}(x) = \mathbf{w}^T x + b$ with ordinary least squares.

Bayesian optimization finds the optimal hyperparameters by minimizing the validation RMSE for one-step prediction:

$$\{\hat{\theta}_{\text{kernel}}, \hat{\sigma}_f, \hat{\sigma}_n, \hat{m}\} = \arg\min_{\theta} \text{RMSE}_{\text{val}}(\theta) \tag{7}$$

this is done for kernel type, length scales $\ell \in [0.1, 20]$, signal variance $\sigma_f^2 \in [0.01, 200]$, noise $\sigma_n^2 \in [10^{-6}, 10^{-1}]$, and mean function type $m \in \{\text{zero}, \text{linear}\}$. For the Periodic kernel, the periodicity is also optimized with $p \in [0.1, 10]$.

4) Systematic Model Development: We found the optimal NARX memory depths with the use of time series cross-validation. Table I shows this for the raw features, with optimal performance at $n_b = 2, n_a = 2$.

TABLE I: Memory depth optimization raw features (RMSE in rad). With na the number of past outputs, and nb number of past inputs

$n_a \backslash n_b$	2	3	4	5
2	**0.0185**	0.0259	0.0313	0.0395
3	0.0234	0.0285	0.0333	0.0411
4	0.0257	0.0312	0.0360	0.0434
5	0.0271	0.0337	0.0405	0.0462

5) Kernel Selection: Bayesian optimization is used with 25 trials to select optimal kernel type and hyperparameters. We optimized for RBF, Matérn (ν=1.5, 2.5), and Periodic kernels, WhiteKernel is always included to prevent overfitting. With length scales [0.1, 20.0], signal variance [0.01, 200.0], and noise variance [10^{-6}, 10^{-1}]. For the periodic kernel the periodicity $p \in [0.1, 10.0]$ is also optimized. Table II gives an overview.

TABLE II: Optimal Parameters for Raw Features GP

Parameter	**Value**
Kernel Type	Periodic (ExpSineSquared)
Mean Function	Linear
Length Scale	0.9754
Constant Value	1.37
Noise Level	0.0043
Periodicity	3.8567
Memory Depth (nb, na)	(2, 2)

Optimal performance was achieved with the Periodic kernel, with period $p = 3.86$ rad, signal variance $\sigma_f^2 = 1.37$, and noise variance $\sigma_n^2 = 1.875 \times 10^{-5}$, while using a linear mean function.

6) Physical Interpretation: While the periodic kernel achieved the lowest validation RMSE, this likely reflects regularities induced by the control strategy rather than inherent system dynamics. The optimized period deviates from the pendulum's natural frequency, indicating the model may be fitting control patterns instead of physical behavior. This

highlights a limitation of purely data-driven kernel selection when physical priors are ignored.

Furthermore, the identified short memory depths ($n_a = n_b = 2$) combined with a relatively small length scale ($\ell = 0.975$ time steps) suggest that the GP predictions rely heavily on immediate past inputs and outputs.

The high signal-to-noise ratio (SNR 230.9), suggests minimal measurement noise. However, such an extremely high SNR also raises concerns about model overfitting and numerical instability. It implies the GP might be overly confident in fitting small-scale data fluctuations rather than genuine underlying dynamics. This could potentially undermine robustness when the model is tested under different conditions with more typical noise levels.

Lastly, the optimality of a linear mean function suggests the presence of a consistent trend in the data that is not well explained by the GP's kernel alone. Including a linear mean does improve predictive performance. However, the model may instead be compensating for systematic errors for example, control bias or artifacts in the experimental setup. In that case, the GP could be modeling side effects of the data collection process rather than the actual system behavior.

B. Results and Validation

1) Model Performance: Table III shows an overview of the performance.

TABLE III: GP-NARX performance metrics raw features

Metric	Test	Benchmark
One-step RMSE (rad)	0.004069	0.00382
Simulation RMSE (rad)	0.102857	0.0271
Overfitting ratio	1.058	–

For one-step prediction, the model achieves a RMSE of around 0.004, comparable to the performance of the "Good NN model" benchmark. The overfitting ratio of 1.06 suggest good generalization. For simulation, the model achieves a RMSE of around 0.1, under-performing compared to the "Good NN model" benchmark.

2) Predictive Capabilities: Multi-step simulation reveals the model's ability to capture long-term dynamics while quantifying uncertainty. Figure 1 demonstrates 500-step ahead predictions with uncertainty bounds.

The simulation RMSE of 0.1029 rad represents approximately 25× degradation from one-step prediction. This result is expected for chaotic underactuated systems. Importantly, the GP's uncertainty estimates grow appropriately with the prediction horizon, providing valuable confidence information for control applications.

TABLE IV: Performance comparison between raw and trigonometric features (RMSE in rad)

Feature Type	Train RMSE	Test RMSE	Simulation RMSE
Raw Features	0.0038	0.0041	0.1029
Trigonometric Features	0.0039	0.0056	0.1117

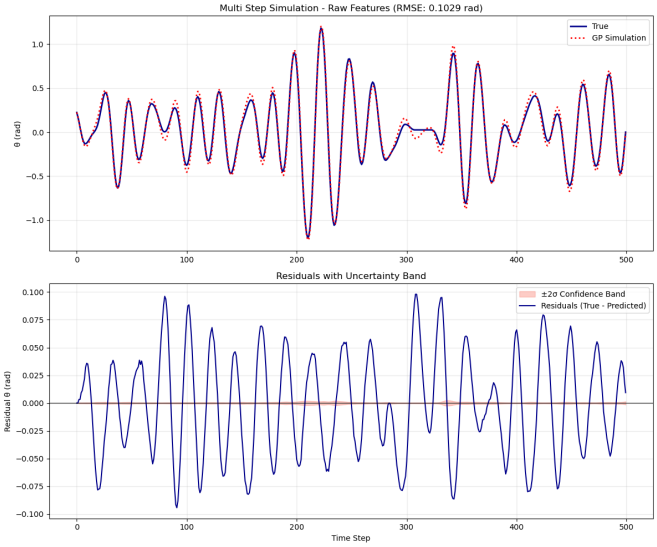


Fig. 1: Multi-step simulation for GP predictions with $\pm 2\sigma$ uncertainty bands. Uncertainty grows over time indicating accumulating prediction errors for the recursive simulation.

3) Feature engineering: Table IV displays a performance comparison between raw and trigonometric features. The results show that using trigonometric features hardly improve the performance. Under optimal conditions, trigonometric features do not seem to be worth the extra computational cost.

III. ANN Model Identification

In this part, the (non-linear) system dynamics will be captured using machine learning techniques. We will use a data-driven system identification methodology as given in Lecture 1. We have selected a NARX model structure and an LSTM model structure to capture the system dynamics. In a NARX model structure, a limited number of previous input and output signals are used to estimate the system dynamics. This state is outside of the ANN. Because LSTM neurons have a learnable memory function, we wanted to know how ANNs with LSTM neurons perform in relation to NARX. In other words, what if the state is embedded in the ANN and is learnable?

A. The Data-Driven Modeling Process

Before training an ANN model on the available data, there are choices to be made. The Data-Driven Modeling Process, as given in Lecture 1 slide 99, has five major choices for the model estimation part.

1) Data related; such as partitioning the given data in training, validation, and test data. 2) Model related; such as selecting the model class and parameters, like model structure, number of layers, and neurons per layer. 3) Cost-related; specifying a cost function to optimize. 4) Optimization method related; such as which optimizer to use and how to use it. Furthermore, 5) a model validation method is required for the estimated model.

B. ANN NARX model estimation

The first step is to define proper choices for the Data-Driven Modeling Process. 1) Data: the data set is divided into a training, validation and test set with a 60-20-20 ratio. Because the data is correlated, it is important to not make the validation and test set too small. Making these data sets too small could result in indirectly overfitting the model on them. 2) Model structure: for simplicity, a homogeneous-layer fully connected neural network is used. The number of layers and number of neurons per layer are determined using hyperparameter tuning. 3) Cost function: when a value has to be predicted, a common choice is the mean squared error (MSE) loss function. As MSE is also the loss function used for benchmarking, this loss function will be used to train (and evaluate) the model. 4) Optimization method: the best optimizer for the problem at hand is determined using hyperparameter tuning. 5) Model validation: the performance of the model on the test set will be determined and compared to given benchmark.

Next, the appropriate hyperparameters for the ANN NARX model will be determined. To limit the search space of a grid search, a sensitivity measurement is performed first. For the sensitivity measurement, a baseline model (with common hyperparameter values) is established and each hyperparameter is adjusted individually. This method gives an indication which hyperparameters are critical and what might be appropriate values for these hyperparameter. First, a proper baseline mode is defined. The baseline model contains 2 layers with 60 neurons each. The learning rate is set to 1×10^{-3} and the weight decay to 1×10^{-6}. The batch size is set to 32. The baseline model is run for 30 epochs and uses Adam as an optimizer. Second, for each hyperparameter some variants are defined. The results of the sensitivity measurement are displayed in table V. From the sensitivity measurement, the following can be concluded. A higher number of layers and a higher number of neurons seem to perform better. The learning rate and weight decay of the baseline model seems to perform best. A batch size of 16 also seems to perform best. A higher number of epochs does seem to increase performance. The Adam optimizer seems to perform better than the RMSprop optimizer. Considering these results while keeping model complexity in mind, a grid search, as defined in table VI, is performed. The grid search results in the following 'optimal' hyperparameters: number of layers = 3, number of neurons = 90, learning rate = 1×10^{-4}, weight decay = 5×10^{-7}, batch size = 16, number of epochs = 40 and optimizer = Adam. The training and validation loss are respectively 1.6×10^{-6} and 2.0×10^{-6}. Implementing and running this model on the test set results in a testing loss of 9.4×10^{-6}. The testing loss corresponds to a RMSE of 3.07×10^{-3} which is lower than the benchmark for a 'Good NN model' of 3.82×10^{-3}. However, when applying the model on the hidden test prediction submission and plotting the results, the model makes some strange predictions. Despite all efforts to prevent it, it looks like the model still indirectly overfitted on the validation and test data. Also, increasing the weight decay, decreasing the amount of nodes and layers, adding noise to the training data and not shuffeling the prepared NARX data strings does not seem to solve the problem. To still get some results, the following simpler structure is defined in Matlab.

TABLE V: Sensitivity measurement for ANN NARX (8 hyperparameters).

Model	Training loss	Validation loss
Baseline	4.45×10^{-6}	5.12×10^{-6}
Number of layers = 1	1.12×10^{-5}	1.143×10^{-5}
Number of layers = 3	2.85×10^{-6}	3.32×10^{-6}
Number of neurons = 30	1.174×10^{-5}	1.261×10^{-5}
Number of neurons = 90	3.03×10^{-6}	3.59×10^{-6}
Learning rate = 1×10^{-4}	5.41×10^{-6}	6.47×10^{-6}
Learning rate = 1×10^{-2}	1.096×10^{-5}	1.212×10^{-5}
Weight decay = 1×10^{-7}	1.896×10^{-5}	1.883×10^{-5}
Weight decay = 1×10^{-5}	1.388×10^{-5}	1.569×10^{-5}
Batch size = 16	2.95×10^{-6}	3.28×10^{-6}
Batch size = 64	3.30×10^{-6}	4.15×10^{-6}
Number of epochs = 20	1.213×10^{-5}	1.245×10^{-5}
Number of epochs = 40	2.09×10^{-6}	2.40×10^{-6}
Optimizer = RMSprop	1.270×10^{-5}	1.650×10^{-5}

TABLE VI: Hyper-parameter grid for ANN NARX (96 grid points).

Hyperparameter	Values explored
Number of layers	{1, 2}
Number of neurons	{60, 90}
Learning rate	$\{5 \times 10^{-5}, 1 \times 10^{-4}, 5 \times 10^{-4}\}$
Weight decay	$\{5 \times 10^{-7}, 1 \times 10^{-6}, 5 \times 10^{-6}\}$
Batch size	16 (fixed)
Number of epochs	{30, 40}
Optimizer	Adam (fixed)

In Matlab we have analyzed one-layer NARX structures for both ReLU and Tansig activation functions. The Tansig activation functions resulted in the best RMSE. To select the optimal number of neurons a Monte-Carlo simulation was run to get statistical data on the networks using the test dataset. The results are shown in Figure 2. Since the variance for $n = 20$ neurons is the first entry with low variance, $n = 20$ is selected as the optimal number of neurons. The requested submission files are generated using a trained 1 layer narx network with Tansig activation functions and 20 neurons in Matlab.

C. ANN LSTM model estimation

First, the five major choices for the Data-Driven Modeling Process must be made. 1) Data; The available data will be divided into 80% for training 10% for validation and 10% for testing. 2) Model structure; For LSTM, There are numerous model structures to explore. However, we will limit the structures to one-layer networks and we will explore the performance of these networks from 4 to 24 neurons. 3) Cost function; we will use a squared error cost function. It is

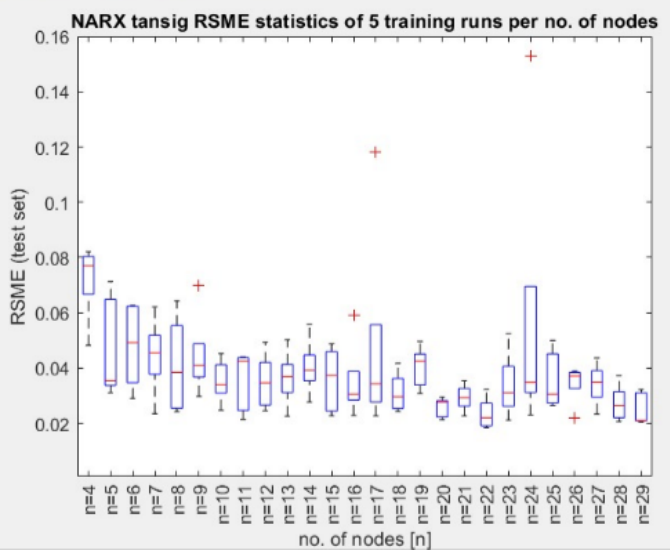


Fig. 2: box plot of RMSE values of the test data set versus the number of neurons n.

common to use a mean squared error cost function; however, a squared error cost function might decrease the individual absolute error. 4) Optimization method: Since LSTM networks are recurrent networks and may become large as the network unfolds during training, we will use the L-BFGS algorithm instead of ADAM. (actually, we started with ADAM but ran into training issues, the cost per epoch was varying a lot, it looked unstable. After switching to L-BFGS, the cost function versus epoch plot was nicely declining.)
Each model is trained using 3000 epochs. Since this LSTM network cannot be trained in open-loop (feedforward) mode, like NARX, training takes more time. (We started with a lower number of epochs, but then the resulting RMSE, when simulating the model, was larger. When using 3000 epochs we had a good trade-off between training time and performance of the ANN).
5) model validation; The performance of a model is expressed in terms of the root mean squared error (RMSE) of the simulated output of the trained ANN and the real measured output, excluding the transient period. A model will be validated using the RMSE results of the training, validation, and test data sets. The procedure from Lecture 1, slide 109 will be used.

1) LSTM transient period: Figure 3 shows the first 200 samples of the ANN output and the actual output. It can be seen that there is a transient period up to the sample number $i = 140$. This means that the ANN requires 140 samples before it reaches steady state. Therefore, the first 140 samples are excluded from the simulation RMSE computation, so the RMSE is computed only over the steady state.

2) LSTM training results: Figure 4 shows the resulting RMSE values after using the trained ANNs for simulation. We also plotted the lower bound for an NARX ANN and the RMSE value of a well-trained NARX ANN.
As can be seen in the figure, the RMSE of both the training

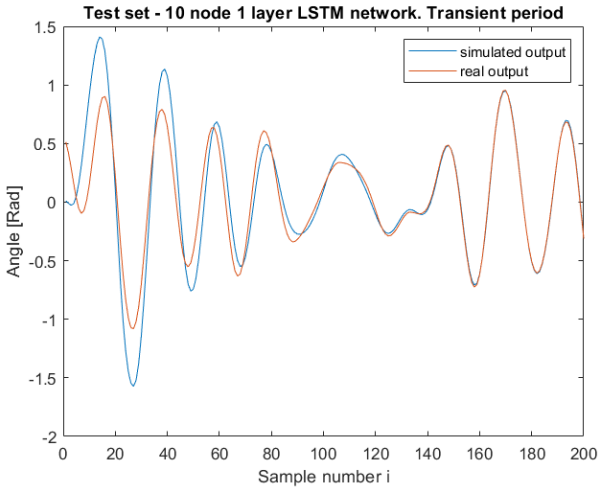


Fig. 3

and test data sets keeps decreasing as the number of neurons increases. This is not the case for the validation set. The RMSE of this set decreases until $n = 10$, and then increases.
To avoid overfitting, selecting $n = 10$ seems the optimal choice for a single layer LSTM network and the given training data. The resulting RMSE of the test data set $= 0.02$ Rad.

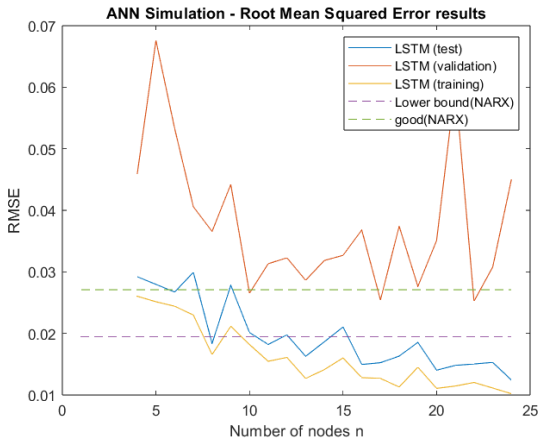


Fig. 4

3) LSTM discussion: The number of neurons in the LSTM ANN is smaller than that of the NARX ANN while having better performance. However, this comes at a computational cost. In case of a NARX ANN with 20 neurons, we have 20 activation functions. A standard LSTM neuron consists of 3 sigmoid and 2 tanh activation functions. So, instead of having 20 activation functions, the 10 node LSTM network has 50 activation functions!
Nevertheless, it is interesting to observe that embedding the state in the ANN results in improved function estimation capabilities after the ANN has seen enough samples.

IV. Reinforcement Learning in Simulation

The goal of this project was to train a control policy capable of reliably swinging up a pendulum from its stable position at the bottom ($\theta = 0$) to the top position ($\theta = \pi$. Due to its underactuated and nonlinear dynamics, effective solutions require balancing exploration for swing-up with precise control near the equilibrium. This makes it ideal for evaluating reinforcement learning (RL) methods, particularly regarding sample efficiency and stability.

We implement and compare both model-free and model-based reinforcement learning strategies. The model-free approaches include DQN, PPO [1], and SAC [2], which directly optimize a control policy from raw interactions with the environment, without requiring a learned model of system dynamics. In contrast, we also implemented PILCO [3], a model-based method that fits a probabilistic Gaussian Process model to transition dynamics and analytically computes expected long-term rewards. This method performs policy optimization within the internalized model and is known for high sample-efficiency. These specific methods were chosen to evaluate how value-based and policy gradient methods differ in sample efficiency, convergence properties, and optimal control capabilities. To assess and gain a better understanding of these methods, we employ hyperparameter tuning and evaluate convergence based on episodes required to consistently reach near-optimal performance.

The state vector used in our experiments is defined as $s_t = [\sin\theta_t, \cos\theta_t, \dot{\theta}_t]^\top \in \mathbb{R}^3$, where θ_t is the disc angle, and $\dot{\theta}_t$ its angular velocity. Representing the angle via its sine and cosine ensures continuity at the periodic boundary $\pm\pi$, avoiding discontinuities inherent in direct angle representations [4]. This approach is beneficial for function approximation when using neural network-based RL methods.

An initial exploration of various reward functions revealed significant practical challenges, including reward hacking, where the agent optimizes the reward through undesirable behaviors such as uncontrolled swinging or being idle in unintended positions. We thus adopted the following simple reward function: $r(\theta_t, u_t) = 0.2 \cdot (1 - \cos\theta_t) - 0.001 \cdot u_t^2$ The function gives an increasingly large bonus near the top position and a small penalty for large control efforts. This resulted in improved convergence stability and reduced pathological behavior compared to sparse or discontinuous rewards. The reward landscape and comparisons between candidate reward functions are illustrated in Fig5, clearly depicting the smoothness and bounded gradients advantageous for policy learning. Other smooth functions, but more complex functions, that are not shown in the graph struggled with balancing rewards for being near the top with penalizing speed and effort, leading to the reward hacking.

A. Deep Q-Network (DQN)

DQN uses the core ideas of value-based RL (bootstrapping, TD updates, experience replay). Unlike simpler table-based Q-learning, DQN can handle high-dimensional state spaces

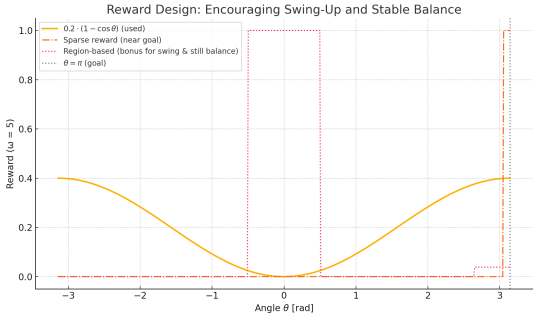


Fig. 5: Reward landscapes of candidate shaping functions across $\theta \in [-\pi, \pi]$.

via function approximation. DQN uses a neural approximator $Q(s, a; \theta)$ to estimate the optimal action-value function $Q^*(s, a)$. It minimizes the squared temporal-difference (TD) error, which is the discrepancy between the current estimate and a one-step lookahead bootstrap target. TD learning updates the value function incrementally based on this error signal.

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right], \tag{8}$$

θ are the parameters of the online Q-network, θ^- are the parameters of the slowly updated target network, $\mathcal{D}$ is a replay buffer that breaks temporal correlations and improves data efficiency. A very small replay buffer may lack diverse experiences, while an overly large one can make learning stale by overusing outdated data; similarly, γ trades off short-term reactivity and long-term planning, where values near 1 favor future rewards more heavily.

We discretize the action space $u \in [-3, 3]$ V into n evenly spaced bins. Ten configurations are formed from the grid in Table VII and trained for 2×10^5 steps.

TABLE VII: Hyper-parameter grid for DQN (10 variants).

Parameter	Values explored
# actions n	$\{5, 10, 15\}$
Mini-batch size B	64 (fixed)
Learning rate η	$\{5 \times 10^{-5}, 10^{-4}\}$
Final exploration $\varepsilon_{\min}$	$\{0.02, 0.1\}$
Replay buffer size	50 000 (fixed)
Target update interval	500 gradient steps

Figure 6 shows the mean episode reward. Every variant reaches an initial plateau at $\overline{R} \approx 58$ by 4×10^4 steps. Where the disc learns the swing-up but is not yet able to correctly balance it. A second growth phase starts around 8×10^4 steps, with the best runs surpassing $R \approx 100$ and stabilising near $R \approx 110$. Which is close to the maximum reward possible, indicating it is able to do an efficient swing-up and keep the disc there.

Overall, the DQN results were remarkably robust across discretization levels. The exploration–exploitation dilemma, where finer grids increase policy precision but slow learning

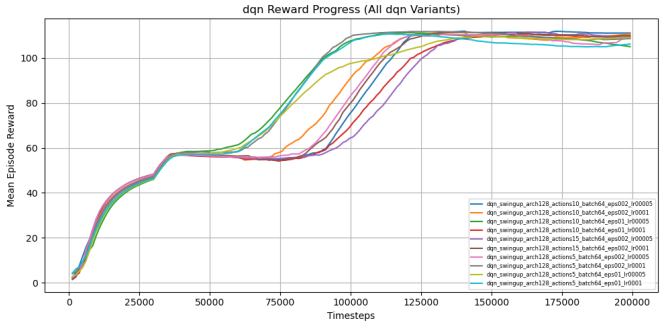


Fig. 6: Mean episode reward vs. timesteps for the ten DQN variants.

due to the larger action space, is visible but not consistently. Even with a limited action space of only 5 actions, the policy is able to maximize the rewards, and the larger action spaces do showcase slower learning but it is also visible that is not the only influence on learning rate. The variant with $\varepsilon_{\min} = 0.1$ sustains higher policy entropy, preventing early convergence to suboptimal policies, whereas $\varepsilon_{\min} = 0.02$ should converge faster once the replay buffer is sufficiently diverse. However as is visible in the orange line, it can also be beneficial to explore diverse actions in such complex tasks and that can even speed up learning. Later `arch128_actions10_batch64_eps002_lr00005` will be compared with the other methods, as it showed the highest final reward with stable training.

B. Proximal Policy Optimisation (PPO)

To benchmark an on-policy actor–critic scheme we implement the PPO algorithm of [5]. PPO alternates between collecting roll-outs with the current stochastic policy $\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta, \sigma_\theta)$ and performing several epochs of minibatch updates that maximise a surrogate objective while constraining the Kullback–Leibler (KL) change in policy. The clipped surrogate that is optimized is

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t\,\hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\big)\Big] \tag{9}$$

$$\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \tag{10}$$

Here, $\hat{A}_t$ denotes the advantage estimate computed using Generalized Advantage Estimation (GAE-λ), and ε is the clip range that constrains the policy update. The policy network includes a separate value head $V_\phi(s)$, which is trained by minimising the squared error to the estimated returns. Additionally, an entropy bonus term $\beta\, H[\pi_\theta]$ is added to the objective to promote exploration by encouraging higher policy stochasticity.

In our hyperparameter sweep, each setting targets a specific trade-off in learning stability, exploration, and function complexity. Other parameters such as rollout length $n_{\text{steps}} = 1024$, minibatch size $B = 512$, and GAE parameter $\lambda = 0.95$ remain unchanged from existing PPO configurations.

TABLE VIII: Hyper-parameter grid for PPO

Parameter	Values explored
Network width h	$\{64, 128\}$ units per layer
Learning rate η	$\{3 \times 10^{-4}, 10^{-3}\}$
Clip range ε	$\{0.1, 0.2\}$
Entropy bonus β	$\{0, 0.01\}$
Roll-out length n_{steps}	1024 (fixed)
Minibatch size B	512 (fixed)
GAE parameter λ	0.95 (fixed)

Figure 7 displays the mean episode reward for all PPO configurations. PPO also shows its ability to maximize rewards, with most runs reaching $R \approx 110$. The architecture width has limited impact: while larger networks ($h = 128$) show slightly faster initial gains, they also introduce higher training variance. The best results are achieved with the more conservative learning rate $\eta = 3 \times 10^{-4}$, particularly when combined with a narrow clip range $\varepsilon = 0.1$, which enforces smaller policy updates and preserves stability. Notably, high learning rate and large clip range combinations (e.g., `arch64_lr0001_clip02_ent001`) occasionally destabilize, as seen in the reward dip near 450k steps. It required 4x as many steps for convergence as DQN, to verify if it is due to the missing replay-buffer or the continuous action space we will also implement the SAC algorithm.

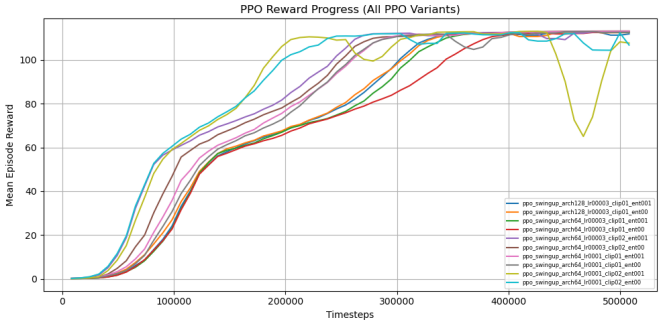


Fig. 7: Mean episode reward vs. timesteps for all PPO variants.

C. Soft Actor-Critic (SAC)

SAC [2], [6] is the third algorithm evaluated, representing an off-policy actor–critic method that optimizes a stochastic policy under an entropy-regularized objective:

$$\mathcal{J} = \sum_t \mathbb{E}_{(s_t,a_t)\sim\pi_\theta}\left[r(s_t,a_t) + \alpha\mathcal{H}(\pi_\theta(\cdot \mid s_t))\right],$$

where $\mathcal{H}$ denotes the differential entropy of the policy distribution. The temperature α balances exploration and exploitation by scaling the entropy term. The policy is parameterized as a diagonal Gaussian, reparameterized for low-variance gradient estimation and trained to maximize the entropy-regularized return. Critics are trained using the Double-Q trick [7], mitigating value overestimation by taking the minimum of two independent estimators. The targets of the critics are updated via Polyak averaging, where a smaller τ (e.g. 0.0005) ensures

stability in high-variance environments. We fix $\gamma = 0.99$ as the discount factor.

TABLE IX: Hyper-parameter grid for SAC.

Parameter	Values explored
Network width h	$\{64, 128\}$
Entropy-target scale η	$\{0.1, 0.2, 0.5\}$
Polyak constant τ	$\{0.0005, 0.002\}$
Learning rate (actor & critics)	3×10^{-4}
Batch size	256
Discount γ	0.99
Replay buffer size	10^6 transitions

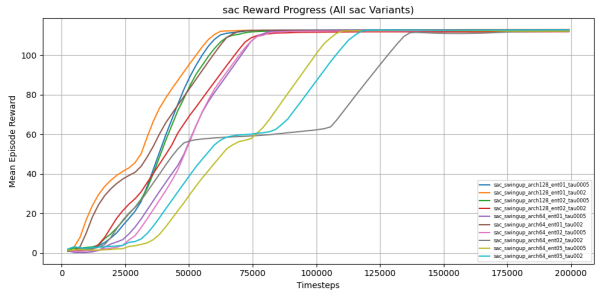


Fig. 8: Mean episode reward vs. timesteps for all SAC variants, shaded region indicates 95% CI over 5 seeds.

Figure 8 shows that all SAC configurations reach the reward plateau ($R \approx 112$) within 8×10^4 steps—roughly four times faster than PPO. The best variant, `arch128_ent01_tau0005`, achieves a final mean reward of 112 ± 2 over the last 5000 steps, sustaining upright balance for approximately 94% of each 300-step episode. Faster target updates led to overshoots in some cases, and later peaks in entropy coefficients ($\eta \geq 0.2$) delayed convergence A. It is clearly visible that SAC benefits from off-policy replay and stable critics, making it sample-efficient and well-suited for continuous control.

D. Conclusion

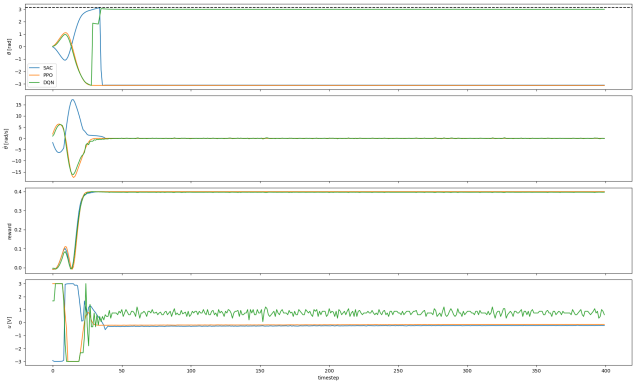


Fig. 9: Final evaluation roll-outs (400 time-steps) for SAC, PPO, and DQN. Top: angular position; centre: angular velocity; bottom: applied voltage.

Figure 9 summarises final controller behavior after tuning. All three controllers successfully swing up and stabilize the disc. in training SAC converged fastest (541 s), because it allowed for parallel environments. PPO followed (1078 s), with similarly smooth signals but requiring 4x as many timesteps. DQN showed higher sample efficiency, but produced less smooth actions due to discretisation but may be preferable where actuators only support discrete voltage levels.

Due to the large training times inherent to RL, not all hyper parameters were exhaustively tested. Careful tuning of these hyper-parameters can shift the training curve. Still, the empirical evidence from this task strongly supports the choice of SAC due to its balance of performance and efficiency. For an even better performance, future work could explore advanced buffer sampling methods such as prioritized experience replay, which assigns higher sampling probabilities to transitions that yield higher TD-errors, thereby accelerating learning. Finally, curriculum learning strategies could also facilitate in the learning process by increasing the difficulty or the weights of part of the reward function. By rewarding angular velocity in the beginning the robot could learn to swing up quicker, reducing this at a later stage will help finding the balancing behavior.

E. Model Internalization-based Reinforcement Learning

In this case, we have used the PILCO policy framework to learn the model using Matlab. The code is based on the pole-cart example from lecture 9.
The PILCO algorithm learns the system dynamics using a Gaussian-processes based model. Learning is done in episodes. In each episode, the system first learns the GP model of the process dynamics using the available data, for example, from previous episodes. Subsequently, model predictive control (MPC) is used to predict the optimal control input sequence u for the entire episode using the GP model, the prediction horizon and the setpoint.
Instead of only applying the first element of the sequence and discarding the rest, as with MPC, the entire sequence is applied to the system. The resulting state of the system is captured to improve the GP model in the next episode.

1) Simulation: For the simulation part, the function "simulate.m" has been modified to have the unbalanced disc dynamics instead of the pole-cart dynamics. The example code uses four state variables for the system dynamics, namely p, v, ω, and θ. The dynamic disc only has state variables ω and θ. Therefore, position p and velocity v set to zero.
The sampling time $T_s = 25$ms and the prediction horizon $n = 40$ steps. The cost function is $1 - \exp(-0.5d^2a)$, with $a > 0$ a scaling factor and d the difference between the current state and the desired state. The desired state $[\omega, \theta] = [0, \pi]$. Since the sampling time of the discrete system is 25 ms, the total learning time for each episode is 1 s. Figure 10 shows the simulation results of the system. It can be seen that after 27 steps, the reference position has been reached and stays at the desired position.

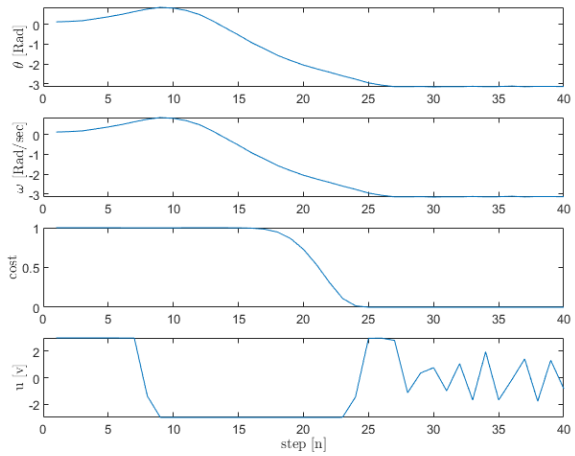


Fig. 10: Simulation results of the learned policy after 5 episodes.

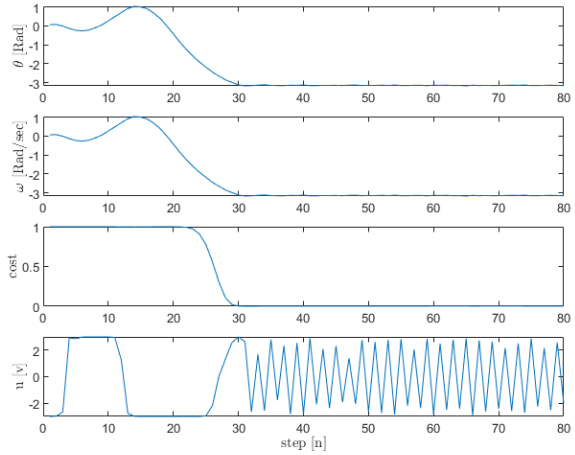


Fig. 11: Experiment results of the learned policy after 5 episodes.

2) Experiment: For the experiment part, the function "simulate.m" has been modified to connect to the FUGIboard directly. Again, position p and velocity v are set to zero. The prediction horizon was increased to $n = 80$ steps to gather more data from the test setup. Figure 11 shows the results of the experiment. It can be seen that after 31 steps, the reference position has been reached and stays at the desired position but is slightly oscillating. The main differences between the simulation and the experiment are 1) it takes longer to reach the desired state. 2) an extra swing has been added to the control policy. This can, for example, be seen in Figure 11 when looking at $n = 1..15$. And 3) Looking at $n > 35$ it can be seen that u requires a larger magnitude to keep the unbalanced disc in position.
This indicates that the test setup probably has more friction than has been modeled in the simulation.

3) Comparison with SAC, PPO and DQN: In Figure 9 the final evaluation roll-outs for SAC, PPO and DQN are shown. PPO reaches the reference point a few time-steps earlier than SAC and DQN. We compare these results with PILCO shown in Figure 11. And we can see that SAC, PPO and DQN use similar policies for swinging up the unbalanced disc. First swing in the opposite direction and then to the setpoint. In case of PILCO, the first swing is in the same direction, then in the opposite direction, and finally to the setpoint. This can also be seen in the behavior of u. Another difference can be seen after reaching the setpoint, PILCO has a less smooth control policy, the control input is alternating between the minimum and maximum value of u. It is not easy to see in the figures, but PPO seems to reach the equilibrium point a few time steps earlier than PILCO, and SAC and DQN just a few time steps after PILCO.

F. Reference-tracking wrapper (SAC only)

The swing-up task is converted into a reference-tracking problem by wrapping the baseline environment `UnbalancedDisk_sincos`. At the start of every episode, the wrapper samples a sinusoidal reference signal:

$$r(t) = A\sin(\omega t + \varphi), \tag{11}$$

$$\varphi \sim \mathcal{U}[0, 2\pi), \qquad A \sim \mathcal{U}[0, A_{\max}], \tag{12}$$

$$\omega = \tfrac{2\pi}{T}, \qquad T \sim \mathcal{U}[0.5T_{\max}, 1.5T_{\max}], \tag{13}$$

with $A_{\max} = 0.26$ rad (approximately 15°) and $T_{\max} = 5$ s. The state observation is augmented as follows:

$$s_t = \left[\sin e_t,\ \cos e_t,\ \dot{\theta}_t,\ A,\ 2\pi/T\right]^\top, e_t = \mathrm{wrap}((\theta_t - \pi) - r(t)),$$

with $\mathrm{wrap}(x) := \mathrm{mod}(x + \pi, 2\pi) - \pi$. Tracking accuracy and actuation cost are balanced using a potential-based shaped reward function similar to the swing-up task:

$$r_t = \cos e_t - 10^{-3} u_t^2,$$

which attains its maximum $r_t = 1$ when perfectly aligned with the reference and discourages large control voltages. During each step, the wrapper computes $r(t)$, evaluates r_t, advances the internal clock $t \leftarrow t + \Delta t$, and returns (s_t, r_t) to SAC. It was trained with the same parameters as the final SAC model and finally the model successfully tracks the reference signal, as depicted in Fig. 12.

G. SAC on the Physical Setup

The SAC controller achieving reliable swing-up in simulation was firstly transferred directly to the physical setup without modification. However, when deployed, Figure 13 captures the initial 10 s of operation, showing that, unlike the smooth swing-up observed in simulation, the disc struggles to balance.

This discrepancy illustrates the sim-to-real gap, arising from possible difference in physics, unmodeled motor dead zones, static friction, and communication delays, all of which bias the policy's value estimates and control decisions.

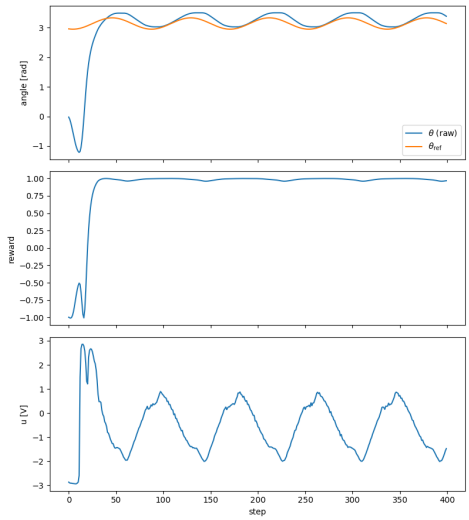


Fig. 12: Results of a reference-tracking run.

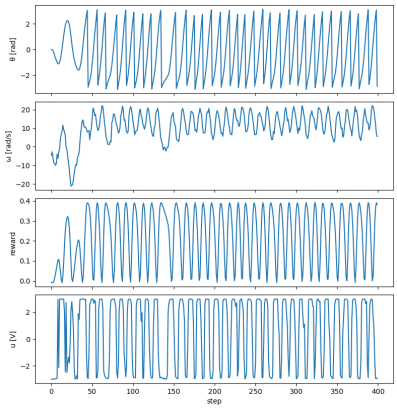
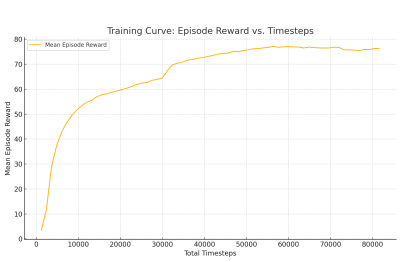


Fig. 13: Initial test on the physical setup using the simulation-trained SAC model.

On hardware, we could not exploit the eight parallel simulation environments, significantly extending the required training time. To reduce waiting time between episodes, a brake function was introduced after each reset. Without this, sometimes 30 seconds would pass before a new episode could start, and even then it started while still swinging in some cases.

Despite extensive retraining, the physical setup did not match simulation performance fully (mean reward dropped from simulation's 112 to approximately 85). It showed inconsistent behavior compared to the simulation. Where in the simulation the swing-up was consistent the whole time, here it only managed to balance the disc half of the time and not fully in the middle. It learned to balance it there, because it was able to apply a more constant, "easy to learn" force, compared to the switching for when balancing in the middle. The model was not able to fully account for the noise that is inherent to a physical set-up. Additional measures could further bridge the sim-to-real gap. By applying domain randomization to the parameters such as masses, friction coefficients, and sensor latency at each episode's start [8] could help policies generalize better by optimizing over a distribution of dynamics. Which will help with later retraining on the physical set-up.

However, when training in simulation with increased observation noise, the model also struggled to converge. This may stem from several factors: increased variance in the TD-targets degrading critic updates, under-regularized policy entropy leading to overly stochastic. If not properly tuned, this can result in either overly cautious behavior or inconsistent, high-variance actions. Combined with limited training time, these effects hinder convergence under noisy conditions.



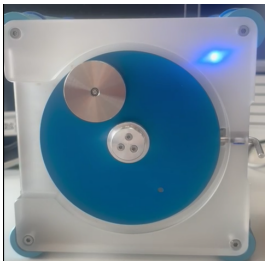


Fig. 14: Left: Training curve from simulation (mean episode reward). Right: Final result on the physical setup for 50% of the runs.

Another significant challenge was the reset function, episodes ending with the disc balanced at the top caused the next episode to initialize from there as the reset function recognizes the disc as standing still. As a result, training became less stable and requires more time to recover from poor episode starts.

This issue compounded existing limitation in hardware training: interaction time is constrained by real-time execution. Consequently, we were unable to properly test a wide variety of models or hyperparameter configurations.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.

[2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.

[3] M. P. Deisenroth and C. E. Rasmussen, "PILCO: A model-based and data-efficient approach to policy search," in *Proceedings of the 28th International Conference on Machine Learning (ICML)*. ICML, 2011, pp. 465–472.

[4] I. Sutskever, G. E. Hinton, and J. Martens, "Recurrent neural networks for text classification with multi-task learning," *arXiv preprint arXiv:0809.0596*, 2008.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[6] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2019.

[7] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.

[8] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.

APPENDIX

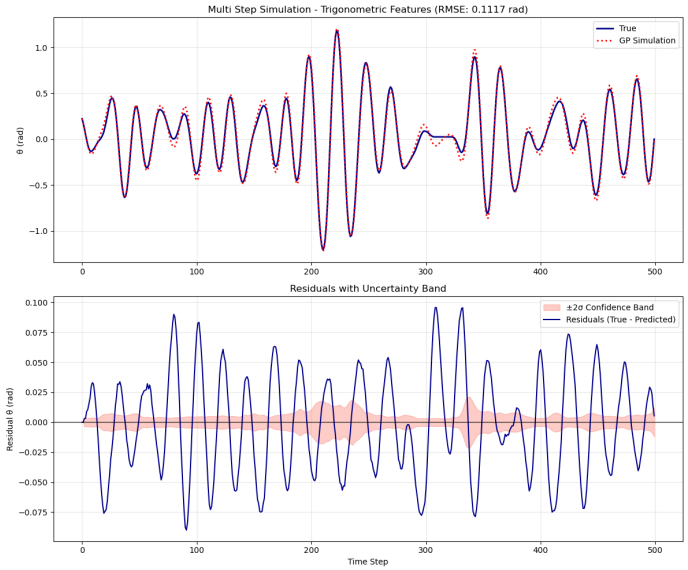


Fig. 15: Multi-step simulation results for trigonometric features.

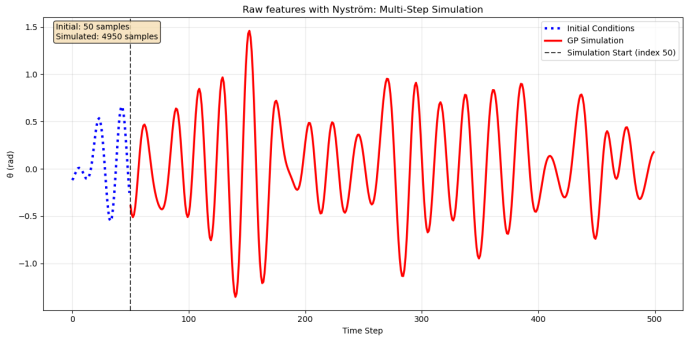


Fig. 16: Multi-step simulation results for raw features with Nyström approximation.

To assess training stability beyond reward curves, we visualise three key signals from SAC training logs:

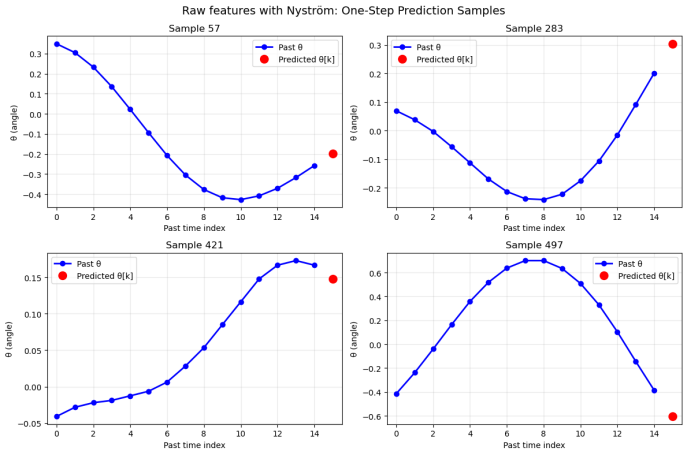


Fig. 17: One-step prediction performance for raw features with Nyström GP. Four representative samples.

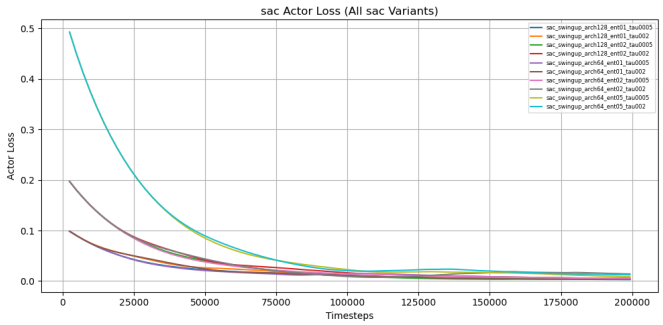


Fig. 18: Actor loss vs. timesteps. All runs exhibit a smooth decline; lower values reflect more confident policies.

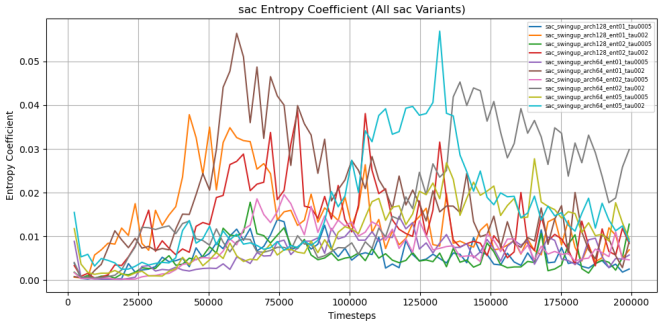


Fig. 20: Learned entropy coefficient α vs. timesteps. Higher η values lead to prolonged exploration (larger α), while $\eta = 0.1$ decays toward 0.01 as policies converge.

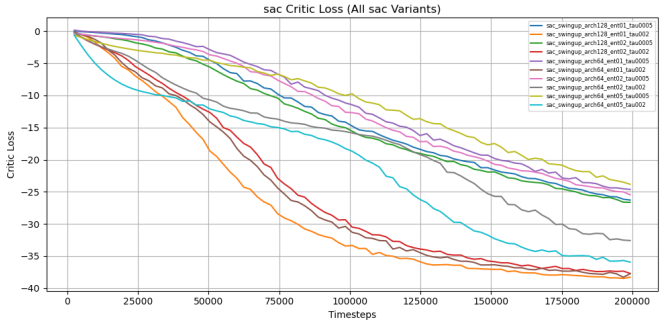


Fig. 19: Critic loss vs. timesteps. A clean monotonic decay indicates stable target tracking. Flattening at later stages may reflect saturation.